

Class 11: Structural Bioinformatics Pt 2

Laura Lu (PID: A17844089)

AlphaFold Data Base (AFDB)

The EBI maintains the largest database of AlphaFold structure prediction models at: <https://alphafold.ebi.ac.uk>

From last class (before Halloween) we saw that PDB had 244,290 (Oct 2025)

The total number of protein sequences in UniProtKB is 199,579,901

Key Point: This is a tiny fraction of sequence space that has structural coverage (0.12%)

$244290 / 199579901 * 100$

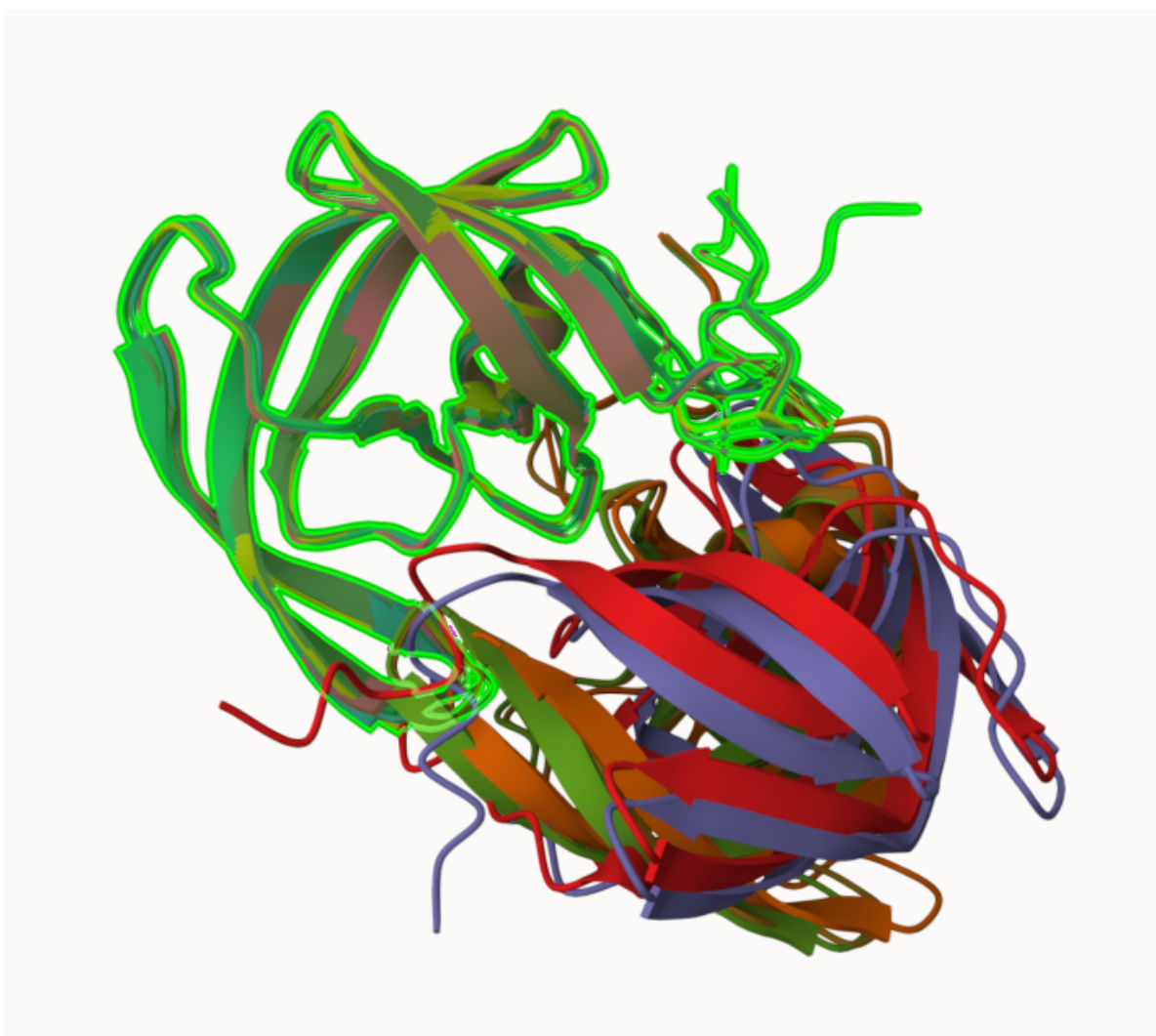
[1] 0.1224021

AFDB is attempting to address this gap...

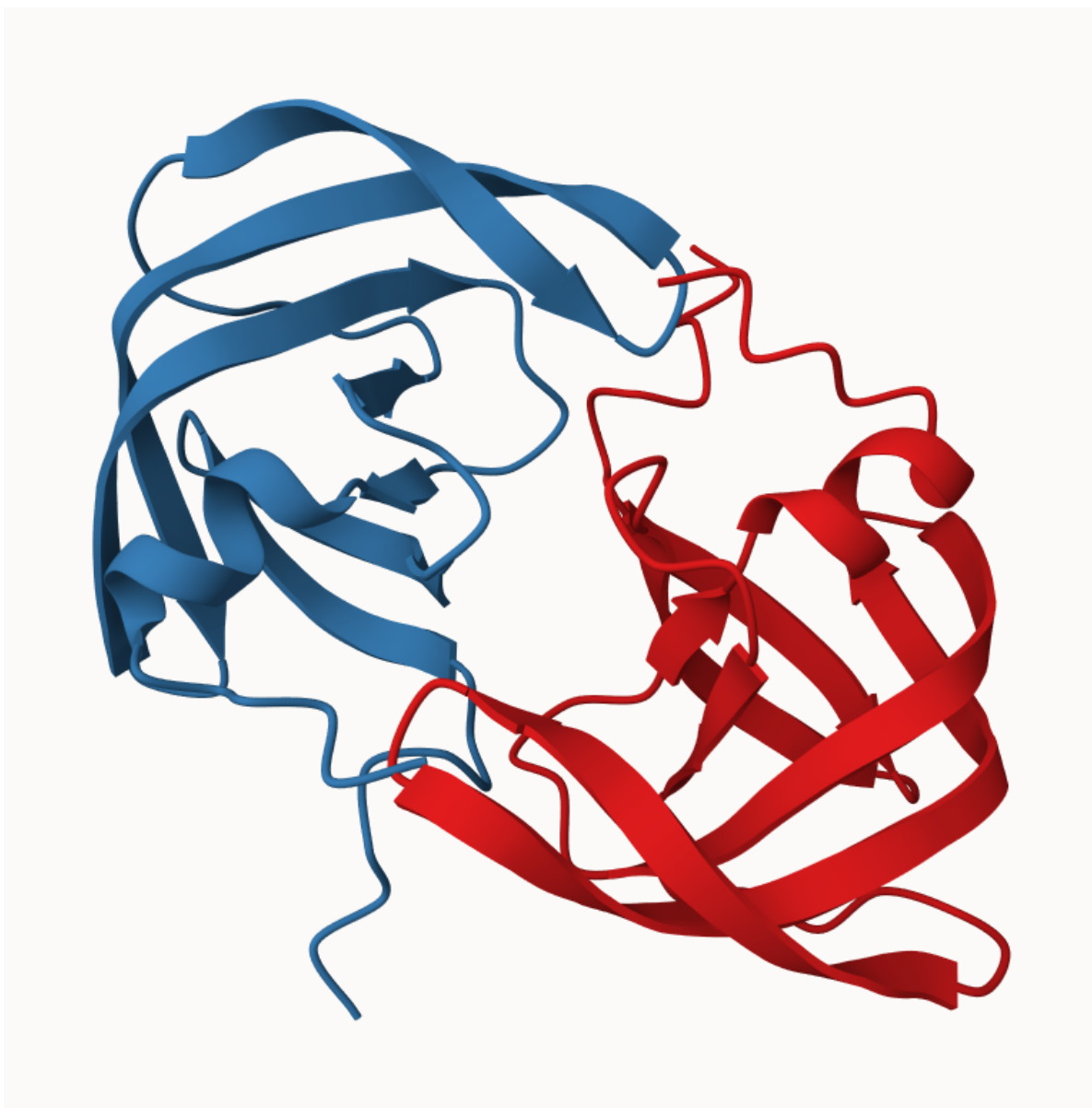
There are two “Quality Scores” from AlphaFold one for residues (i.e each amino acid) called **pLDDT** score. The other **PAE** score measures the confidence in the relative position of two residues (i.e. a score for every pair of residues).

Generating your own structure predictions

Figure of 5 generated HIV-PR models



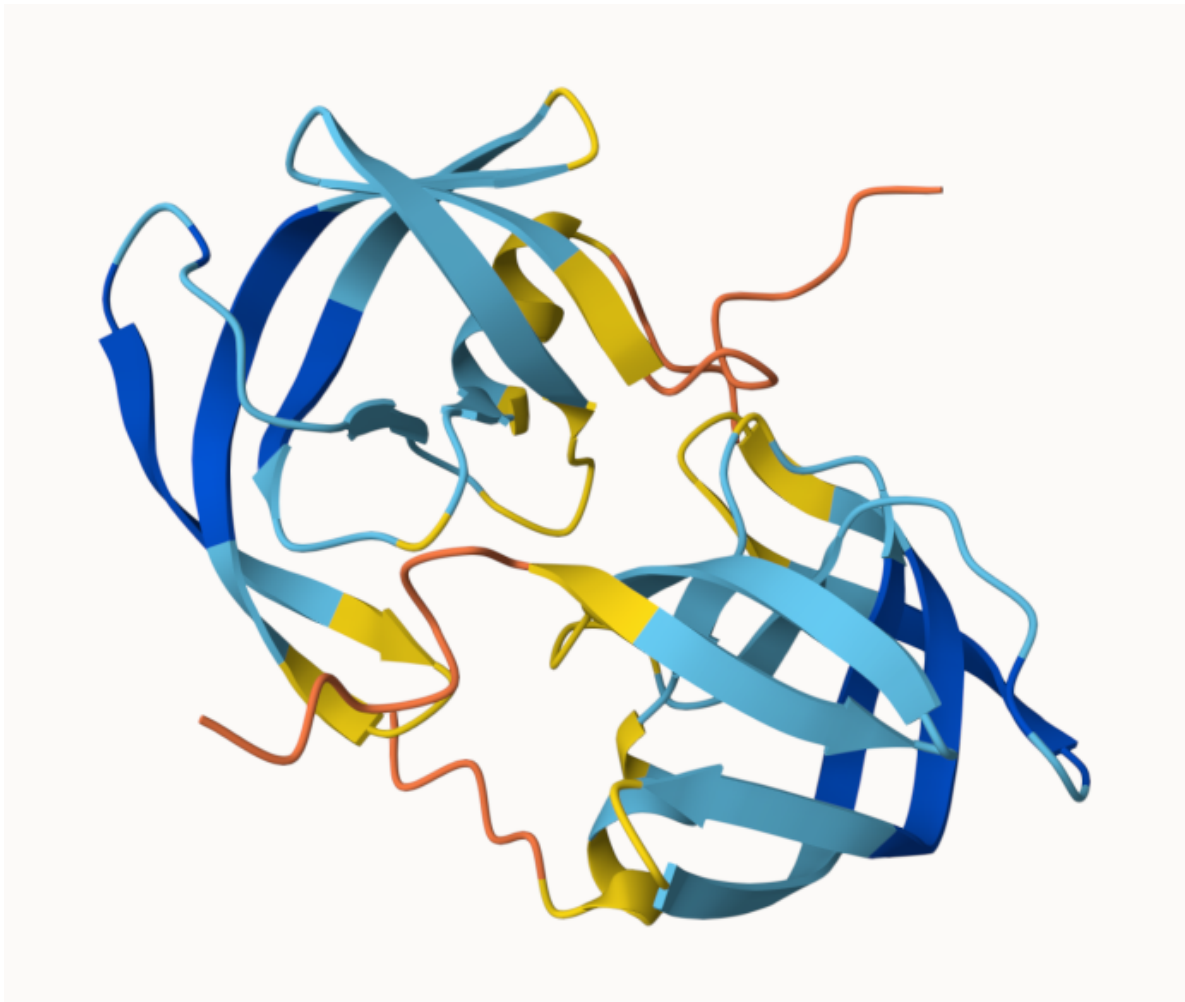
and the top ranked model colored by chain.



pLDDT score for model 1



and model 5



Custom analysis of resulting models in R

Read key result files into R. The first thing I need to know is what my results directory/folder is called (i.e. its name is different for every AlphaFold run/job)

```
results_dir <- "hivprdimer_23119:"  
# File names for all PDB models  
pdb_files <- list.files(path=results_dir,  
                        pattern=".pdb",  
                        full.names = TRUE)
```

```
# Print our PDB file names
basename(pdb_files)
```

```
[1] "HIVPR_dimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb"
[2] "HIVPR_dimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000.pdb"
[3] "HIVPR_dimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000.pdb"
[4] "HIVPR_dimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000.pdb"
[5] "HIVPR_dimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb"
```

```
library(bio3d)
```

```
m1 <- read.pdb(pdb_files[1])
m1
```

```
Call: read.pdb(file = pdb_files[1])
```

```
Total Models#: 1
```

```
Total Atoms#: 1514, XYZs#: 4542 Chains#: 2 (values: A B)
```

```
Protein Atoms#: 1514 (residues/Calpha atoms#: 198)
```

```
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)
```

```
Non-protein/nucleic Atoms#: 0 (residues: 0)
```

```
Non-protein/nucleic resid values: [ none ]
```

```
Protein sequence:
```

```
PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
VNIIGRNLLTQIGCTLNF
```

```
+ attr: atom, xyz, calpha, call
```

```
# Read all data from Models
# and superpose/fit coords
pdbs <- pdbaln(pdb_files, fit=TRUE, exefile="msa")
```

```
Reading PDB files:
```

```
hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb
```

```

hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_0
hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_0
hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_0
hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_0
.....

```

Extracting sequences

```

pdb/seq: 1   name: hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_001_alphafold2_multimer
pdb/seq: 2   name: hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_002_alphafold2_multimer
pdb/seq: 3   name: hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_003_alphafold2_multimer
pdb/seq: 4   name: hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_004_alphafold2_multimer
pdb/seq: 5   name: hivprdimer_23119:/HIVPR_dimer_23119_unrelaxed_rank_005_alphafold2_multimer

```

```
rd <- rmsd(pdbbs, fit=T)
```

Warning in rmsd(pdbbs, fit = T): No indices provided, using the 198 non NA positions

```
range(rd)
```

```
[1] 0.000 14.929
```

```

# Read a reference PDB structure
pdb <- read.pdb("1hsg")

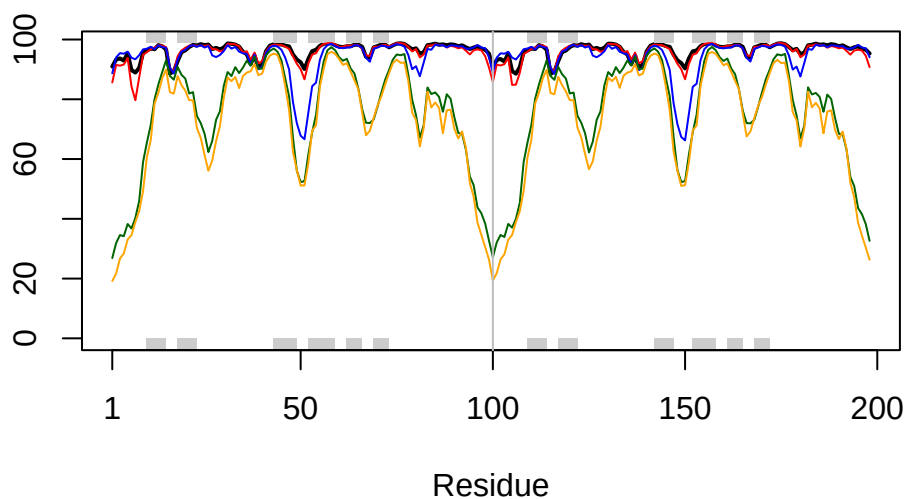
```

Note: Accessing on-line PDB file

```

plotb3(pdbbs$b[1,], typ="l", lwd=2, sse=pdb)
points(pdbbs$b[2,], typ="l", col="red")
points(pdbbs$b[3,], typ="l", col="blue")
points(pdbbs$b[4,], typ="l", col="darkgreen")
points(pdbbs$b[5,], typ="l", col="orange")
abline(v=100, col="gray")

```



```
core <- core.find(pdb)
```

```
core size 197 of 198 vol = 8265.928
core size 196 of 198 vol = 1535.897
core size 195 of 198 vol = 986.57
core size 194 of 198 vol = 909.703
core size 193 of 198 vol = 840.193
core size 192 of 198 vol = 793.655
core size 191 of 198 vol = 754.101
core size 190 of 198 vol = 717.868
core size 189 of 198 vol = 684.463
core size 188 of 198 vol = 652.166
core size 187 of 198 vol = 622.743
core size 186 of 198 vol = 597.709
core size 185 of 198 vol = 568.568
core size 184 of 198 vol = 541.299
core size 183 of 198 vol = 517.214
core size 182 of 198 vol = 496.66
core size 181 of 198 vol = 477.214
core size 180 of 198 vol = 458.812
core size 179 of 198 vol = 442.685
core size 178 of 198 vol = 418.75
```


core size 177 of 198	vol = 405.786
core size 176 of 198	vol = 392.05
core size 175 of 198	vol = 378.57
core size 174 of 198	vol = 366.937
core size 173 of 198	vol = 353.608
core size 172 of 198	vol = 344.791
core size 171 of 198	vol = 332.816
core size 170 of 198	vol = 322.224
core size 169 of 198	vol = 312.69
core size 168 of 198	vol = 297.322
core size 167 of 198	vol = 282.977
core size 166 of 198	vol = 274.468
core size 165 of 198	vol = 265.976
core size 164 of 198	vol = 257.946
core size 163 of 198	vol = 249.169
core size 162 of 198	vol = 238.767
core size 161 of 198	vol = 232.897
core size 160 of 198	vol = 225.69
core size 159 of 198	vol = 220.914
core size 158 of 198	vol = 214.066
core size 157 of 198	vol = 208.998
core size 156 of 198	vol = 201.995
core size 155 of 198	vol = 197.813
core size 154 of 198	vol = 191.188
core size 153 of 198	vol = 184.362
core size 152 of 198	vol = 178.256
core size 151 of 198	vol = 173.136
core size 150 of 198	vol = 165.806
core size 149 of 198	vol = 160.026
core size 148 of 198	vol = 157.055
core size 147 of 198	vol = 150.123
core size 146 of 198	vol = 144.781
core size 145 of 198	vol = 138.674
core size 144 of 198	vol = 133.21
core size 143 of 198	vol = 127.588
core size 142 of 198	vol = 123.674
core size 141 of 198	vol = 117.858
core size 140 of 198	vol = 112.261
core size 139 of 198	vol = 108.414
core size 138 of 198	vol = 104.277
core size 137 of 198	vol = 100.881
core size 136 of 198	vol = 96.595
core size 135 of 198	vol = 93.274

core size 134 of 198	vol = 92.168
core size 133 of 198	vol = 88.258
core size 132 of 198	vol = 83.067
core size 131 of 198	vol = 79.788
core size 130 of 198	vol = 75.252
core size 129 of 198	vol = 70.613
core size 128 of 198	vol = 67.31
core size 127 of 198	vol = 64.081
core size 126 of 198	vol = 61.608
core size 125 of 198	vol = 59.449
core size 124 of 198	vol = 57.323
core size 123 of 198	vol = 54.705
core size 122 of 198	vol = 52.677
core size 121 of 198	vol = 50.588
core size 120 of 198	vol = 48.223
core size 119 of 198	vol = 45.776
core size 118 of 198	vol = 43.587
core size 117 of 198	vol = 41.969
core size 116 of 198	vol = 39.975
core size 115 of 198	vol = 37.457
core size 114 of 198	vol = 35.505
core size 113 of 198	vol = 33.764
core size 112 of 198	vol = 31.714
core size 111 of 198	vol = 29.75
core size 110 of 198	vol = 28.915
core size 109 of 198	vol = 26.991
core size 108 of 198	vol = 25.05
core size 107 of 198	vol = 23.505
core size 106 of 198	vol = 21.805
core size 105 of 198	vol = 20.109
core size 104 of 198	vol = 18.511
core size 103 of 198	vol = 16.373
core size 102 of 198	vol = 15.393
core size 101 of 198	vol = 13.322
core size 100 of 198	vol = 12.423
core size 99 of 198	vol = 10.77
core size 98 of 198	vol = 10.314
core size 97 of 198	vol = 8.903
core size 96 of 198	vol = 7.455
core size 95 of 198	vol = 5.843
core size 94 of 198	vol = 4.517
core size 93 of 198	vol = 3.186
core size 92 of 198	vol = 2.696

```
core size 91 of 198  vol = 2.119
core size 90 of 198  vol = 1.701
core size 89 of 198  vol = 1.259
core size 88 of 198  vol = 0.862
core size 87 of 198  vol = 0.636
core size 86 of 198  vol = 0.526
core size 85 of 198  vol = 0.448
FINISHED: Min vol ( 0.5 ) reached
```

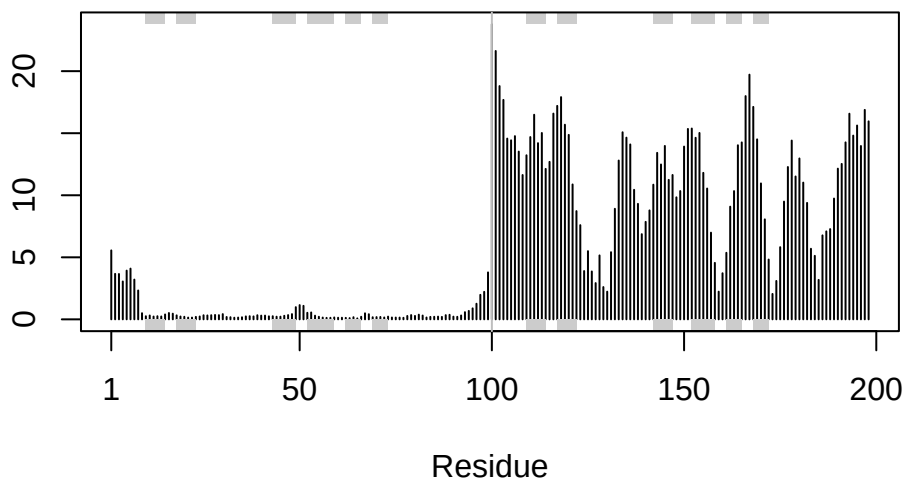
```
core.inds <- print(core, vol=0.5)
```

```
# 86 positions (cumulative volume <= 0.5 Angstrom^3)
  start end length
1      8 48      41
2     50 50       1
3     52 95      44
```

```
xyz <- pdbfit(pdb, core.inds, outpath="corefit_structures")
```

```
rf <- rmsf(xyz)
```

```
plotb3(rf, sse=pdb)
abline(v=100, col="gray", ylab="RMSF")
```

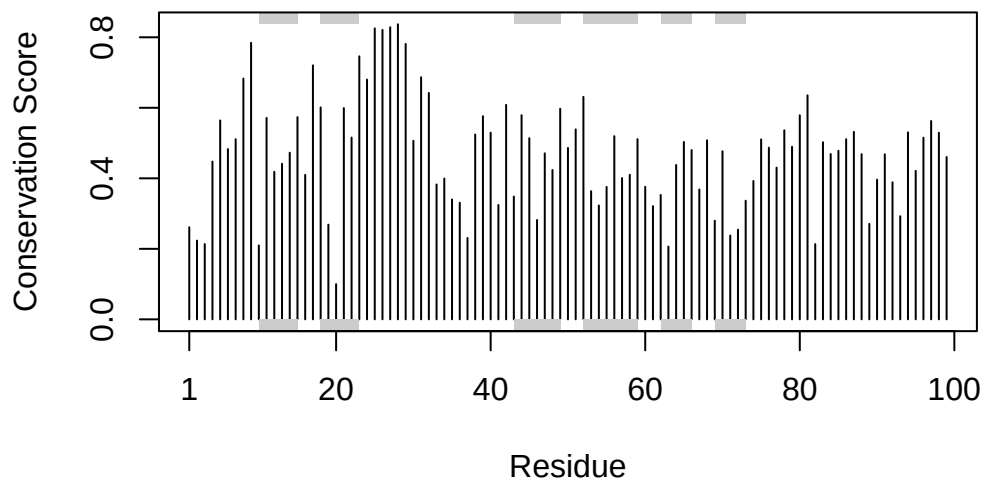


```
# Read alignment
aln_file <- list.files(path = results_dir,
                       pattern = ".a3m$",
                       full.names = TRUE)
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
[1] " ** Duplicated sequence id's: 101 **"
[2] " ** Duplicated sequence id's: 101 **"
```

```
sim <- conserv(aln)
```

```
plotb3(sim[1:99],
        sse=trim.pdb(pdb, chain="A"),
        ylab="Conservation Score")
```



```
head(m1$atom)
```

	type	eleno	elety	alt	resid	chain	resno	insert	x	y	z	o	b
1	ATOM	1	N	<NA>	PRO	A	1	<NA>	16.938	-3.990	-6.129	1	90.94
2	ATOM	2	CA	<NA>	PRO	A	1	<NA>	16.938	-2.557	-6.430	1	90.94
3	ATOM	3	C	<NA>	PRO	A	1	<NA>	16.438	-1.707	-5.266	1	90.94
4	ATOM	4	CB	<NA>	PRO	A	1	<NA>	15.992	-2.449	-7.629	1	90.94
5	ATOM	5	O	<NA>	PRO	A	1	<NA>	15.836	-2.232	-4.324	1	90.94
6	ATOM	6	CG	<NA>	PRO	A	1	<NA>	15.070	-3.623	-7.496	1	90.94

	segid	elesy	charge
1	<NA>	N	<NA>
2	<NA>	C	<NA>
3	<NA>	C	<NA>
4	<NA>	C	<NA>
5	<NA>	O	<NA>
6	<NA>	C	<NA>

```
m1$atom$b[m1$calpha]
```

```
[1] 90.94 93.31 93.69 92.94 95.38 89.75 88.94 90.50 95.50 96.31 97.00 96.75
[13] 98.12 97.75 96.69 88.62 89.31 92.75 95.75 96.38 97.50 98.19 98.00 98.44
[25] 98.12 98.31 96.81 97.19 96.69 97.31 98.69 98.56 98.31 97.62 95.88 95.06
```

```

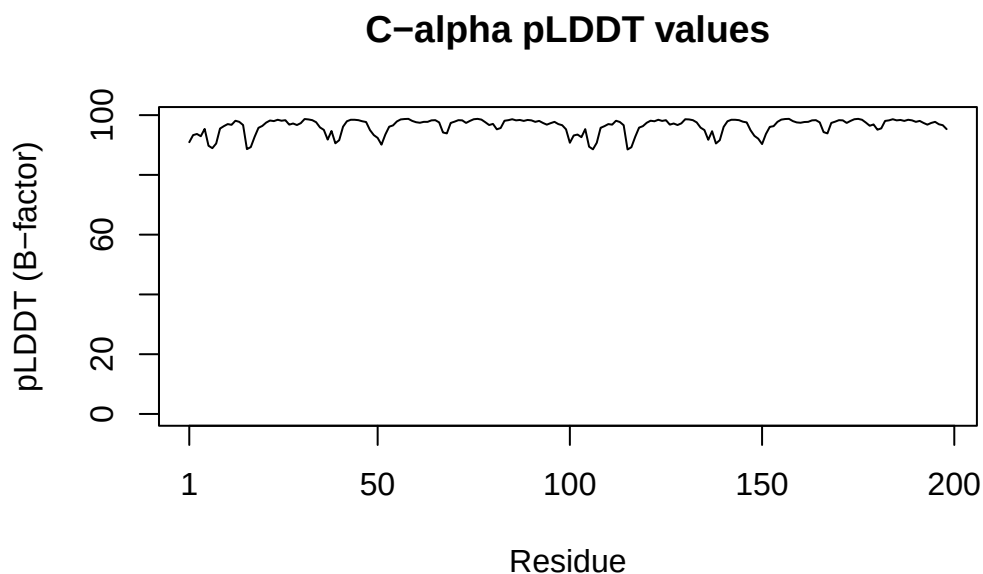
[37] 91.81 94.69 90.56 91.62 96.12 97.94 98.44 98.44 98.31 97.94 97.69 95.00
[49] 93.31 92.38 90.12 93.50 96.12 96.56 97.88 98.56 98.69 98.75 98.06 97.62
[61] 97.44 97.75 97.75 98.25 98.31 97.56 94.19 93.88 97.38 97.81 98.31 98.25
[73] 97.38 98.06 98.62 98.75 98.50 97.62 96.69 97.06 95.25 95.69 98.12 98.31
[85] 98.62 98.25 98.38 98.06 98.38 98.25 97.75 98.06 97.38 96.81 97.31 97.75
[97] 97.00 96.62 95.25 90.75 93.19 93.50 92.62 95.31 89.44 88.56 90.75 95.75
[109] 96.31 97.00 96.81 98.12 97.69 96.62 88.50 89.25 92.75 95.81 96.31 97.44
[121] 98.12 97.94 98.44 98.06 98.31 96.81 97.19 96.69 97.25 98.62 98.56 98.31
[133] 97.56 95.81 95.00 91.75 94.62 90.50 91.62 96.06 97.94 98.44 98.44 98.31
[145] 97.81 97.56 94.94 93.06 92.12 90.31 93.69 96.06 96.31 97.75 98.50 98.69
[157] 98.75 98.00 97.56 97.44 97.69 97.75 98.25 98.31 97.56 94.31 93.88 97.38
[169] 97.81 98.31 98.25 97.38 98.06 98.62 98.75 98.44 97.50 96.44 96.88 95.12
[181] 95.56 98.06 98.25 98.62 98.25 98.38 98.06 98.44 98.25 97.75 98.06 97.38
[193] 96.81 97.38 97.75 96.94 96.62 95.31

```

```

plotb3(m1$atom$b[m1$calpha],
      typ = "l",
      ylab = "pLDDT (B-factor)",
      main = "C-alpha pLDDT values")

```



Residue conservation from alignmnet file

```
aln_file <- list.files(path=results_dir,  
                      pattern=".a3m$",  
                      full.names = TRUE)
```

Read this into R

```
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
[1] " ** Duplicated sequence id's: 101 **"  
[2] " ** Duplicated sequence id's: 101 **"
```

How many sequences are in this alignment

```
dim(aln$ali)
```

```
[1] 5397 132
```

We can score residue conservation in the alignment with the `conserv()` function.

```
sim <- conserv(aln)
```

```
con <- consensus(aln, cutoff = 0.9)  
con$seq
```

```
[1] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"  
[19] "-" "-" "-" "-" "-" "-" "D" "T" "G" "A" "-" "-" "-" "-" "-" "-" "-"  
[37] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"  
[55] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"  
[73] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"  
[91] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"  
[109] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"  
[127] "-" "-" "-" "-" "-" "-"
```

```
m1.pdb <- read.pdb(pdb_files[1])  
occ <- vec2resno(c(sim[1:99], sim[1:99]), m1.pdb$atom$resno)  
write.pdb(m1.pdb, o=occ, file="m1_conserv.pdb")
```